

# Ismail’s Glossary

## A Complete Navigation Index for Mathlib4

Muhammed Ismail  
Independent Researcher  
Email: [literacity@outlook.com](mailto:literacity@outlook.com)  
ORCID: [0009-0000-3713-7105](https://orcid.org/0009-0000-3713-7105)

July 2026

This paper describes a resource, not a result: a complete, human-readable navigation index for every one of Mathlib4’s 9,150 modules — the mathematics library of the Lean 4 proof assistant, and by most measures one of the largest and most active libraries of formalized mathematics anywhere. The problem it solves is narrow but persistent. Every existing way of searching Mathlib — a declaration-name lookup, an in-editor tactic, a semantic search engine — presupposes that the user already knows, at least approximately, what they are looking for. None of them answer the question a newcomer actually asks: *I want to work with spectral sequences; where do I even begin?* This glossary answers exactly that question, one module at a time, in plain mathematical English rather than Lean’s internal vocabulary, explicitly marking how each module differs from its similarly named neighbors. What results is not a partial survey but a complete one: every directory and every file, all 9,150 of them, described, reviewed, and pinned to a fully specified reference snapshot. The glossary is released as five deliverables — a structured JSON file, a retrieval-augmented-generation export, a self-contained Claude Skill, a community-maintained spreadsheet, and an interactive website with both a searchable glossary tree and a dedicated visual atlas of the library’s shape — as an independent, open-source resource for the Lean and Mathlib community, not an official product of either.

*Author’s note.* The glossary text, the data model, and the supporting tooling described in this paper are the sole original work of the author — written, checked, and maintained without co-authors, institutional affiliation, or external funding. The complete project is available at <https://github.com/M-Ismail-ZA/IsmailsGlossary>.

*Independence note.* Ismail’s Glossary is an independent, community-created resource. It is not an official product of the Lean Focused Research Organization (Lean FRO) or the Mathlib community, is not maintained by either, and its content has not been formally reviewed by Mathlib’s maintainers. All descriptive claims about Mathlib4’s structure and content reflect the reference snapshot fixed in Section 2.

**Keywords:** Lean 4; Mathlib; formal mathematics; proof assistants; retrieval-augmented generation; open educational resources; software documentation

## Plain-Language Overview

Imagine handing someone a library with no catalogue: the books are well written, correctly shelved by a logic that made sense to whoever shelved them, and almost entirely undiscoverable to anyone else. That is what it feels like to approach Mathlib4 for the first time. Mathlib is enormous — 9,150 directories and files, formalizing everything from group theory to measure theory to algebraic geometry — and every piece of it is real, checked, working mathematics. What it has never had is a catalogue: something that tells a newcomer, in their own words, what is on which shelf.

**What did I build?** A description, written in plain mathematical English, for every single one of Mathlib’s 9,150 modules — not a sample, not the important-looking ones, all of them — stating what each contains and, where it matters, how it differs from a similarly named neighbor two shelves over. I packaged that catalogue five different ways: as raw data for any program that wants it, as a search tool built into Claude, as a spreadsheet the community can keep editing, and as a website with both a search box and a zoomable map of the whole library.

**Why does this matter to anyone who has never heard of Lean?** Because the underlying problem is not really about Lean. It is the problem every large, well-organized, badly-indexed body of knowledge eventually faces: the people who built it can find anything in seconds, and everyone else is locked out by the same organization that makes it powerful. A high-school student curious about modern mathematics, a graduate student starting a first formalization project, and an AI system trying not to invent a file that does not exist are all, in the end, asking the same catalogue for the same kind of help.

**Is it finished, or a permanent work in progress?** Both, on purpose. Every module is described for a specific, dated, named snapshot of Mathlib — so the glossary is genuinely complete, not partially filled in and improving forever. Mathlib itself keeps growing past that snapshot, so the project is also built to be forked, versioned, and extended by anyone who needs the next one.

Everything past this point is the detailed version of the same three questions — what was built, how completely, and how it stays current — stated precisely enough to be checked against the released data rather than taken on trust.

# Contents

|           |                                                                 |           |
|-----------|-----------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                                             | <b>3</b>  |
| <b>2</b>  | <b>Background: Lean 4, Mathlib4, and the Navigation Problem</b> | <b>3</b>  |
| 2.1       | What Lean 4 Is                                                  | 3         |
| 2.2       | Mathlib4 and Its Scale                                          | 4         |
| 2.3       | Existing Tools and Their Limitations                            | 4         |
| <b>3</b>  | <b>What Was Built: Five Deliverables</b>                        | <b>5</b>  |
| 3.1       | The Glossary JSON (Primary Data File)                           | 5         |
| 3.2       | The RAG JSON (Embedding-Ready Export)                           | 6         |
| 3.3       | The Claude Skill                                                | 6         |
| 3.4       | The Master Spreadsheet                                          | 6         |
| 3.5       | The Interactive Website                                         | 6         |
| <b>4</b>  | <b>The Glossary Data: Design and Coverage</b>                   | <b>7</b>  |
| 4.1       | Complete Coverage for the Reference Snapshot                    | 7         |
| 4.2       | The ‘Distinct From’ Design Philosophy                           | 7         |
| 4.3       | Plain-Language Searchability                                    | 8         |
| 4.4       | Benchmark Theorem Entries                                       | 8         |
| <b>5</b>  | <b>The Claude Skill: Structured Retrieval for AI</b>            | <b>8</b>  |
| 5.1       | The Problem It Solves                                           | 8         |
| 5.2       | How the Skill Works                                             | 8         |
| 5.3       | Anti-Hallucination by Design                                    | 9         |
| 5.4       | Installation and Usage                                          | 9         |
| <b>6</b>  | <b>AI and LLM Integration: Beyond Claude</b>                    | <b>9</b>  |
| 6.1       | The Open JSON on GitHub                                         | 9         |
| 6.2       | Retrieval-Augmented Generation                                  | 9         |
| 6.3       | Fine-Tuning and Instruction Datasets                            | 10        |
| 6.4       | Integration with AI Proof Assistants                            | 10        |
| <b>7</b>  | <b>GitHub Repository Structure</b>                              | <b>10</b> |
| <b>8</b>  | <b>Licensing, Citation, and Availability</b>                    | <b>11</b> |
| 8.1       | License                                                         | 11        |
| 8.2       | How to Cite This Work                                           | 11        |
| 8.3       | Availability                                                    | 11        |
| <b>9</b>  | <b>Accessibility and Educational Philosophy</b>                 | <b>11</b> |
| 9.1       | Designed for All Skill Levels                                   | 11        |
| 9.2       | Formalization Literacy as Early Education                       | 12        |
| 9.3       | The Mathematical Divergence Perspective                         | 12        |
| 9.4       | Quick Copy: From Reading to Proving                             | 12        |
| <b>10</b> | <b>Community Model: A Living Document</b>                       | <b>13</b> |
| 10.1      | Versioning: Three Numbers, Not One                              | 13        |
| 10.2      | Complete at the Reference Snapshot                              | 13        |
| 10.3      | Forking for Exact Reproducibility                               | 13        |
| 10.4      | Contribution Workflow                                           | 13        |
| 10.5      | Mathlib Grows, the Glossary Follows                             | 13        |
| <b>11</b> | <b>Significance and Impact</b>                                  | <b>14</b> |
| 11.1      | For Mathlib Contributors and Users                              | 14        |
| 11.2      | For Mathematics Education                                       | 14        |

|                                              |    |
|----------------------------------------------|----|
| 11.3 For AI-Assisted Formalization . . . . . | 14 |
| 11.4 For Independent Research . . . . .      | 14 |
| <b>12 Conclusion</b> . . . . .               | 14 |
| <b>References</b> . . . . .                  | 15 |

# 1 Introduction

Ismail’s Glossary is a complete, human-readable navigation index for Mathlib4 — the community-maintained mathematics library built on the Lean 4 proof assistant, and one of the largest and most active libraries of formalized mathematics in existence (Baanen et al., 2026). The project describes every one of the 9,150 addressable modules in the Mathlib4 hierarchy — 1,129 directories and 8,021 source files — each carrying a carefully written one-to-three sentence description explaining what the module contains, what it is used for, and how it differs from similar-sounding neighbors.

The glossary is fully complete for its reference snapshot: Lean 4.29.1, Mathlib4 **master** as of 2026-05-09 (commit `1ad783f9bf`). Completeness here means what it says: not a majority of modules, not the popular ones, all of them. This snapshot serves as a stable, citable baseline — anyone wishing to work immediately can fork the repository at this exact point without waiting for updates. From this foundation the project becomes a living document, updated as Mathlib grows, with contributions tracked through a moderated community workflow (Section 10).

Five deliverables are published together in a single public GitHub repository:

- A **structured JSON file** — the full glossary in machine-readable form, ingestible by any AI platform or retrieval pipeline.
- A companion **RAG JSON** — a flat, embedding-ready export for developers building retrieval-augmented generation systems who want a drop-in data source.
- A **Claude Skill** — a self-contained bundle that lets Claude answer Mathlib navigation questions accurately using the glossary as a structured reference, rather than relying on frozen training-time knowledge.
- A **master spreadsheet** — the authoritative contributor workbook, shared via Google Sheets for community additions under moderated review, with a static snapshot published alongside it.
- An **interactive website** — featuring a searchable, browsable glossary tree, a dedicated visual atlas of the library’s structure, a Lean 4 syntax reference, and a step-by-step environment setup guide.

None of these five is a substitute for another; Section 3 treats each as answering a different question for a different reader. A high-school student curious about modern mathematics can browse the website as a structured survey of the field. An experienced Lean contributor can use the Claude Skill as a pre-flight navigator before writing proofs. An AI tool builder can use the JSON as a ready-made, hallucination-resistant knowledge base for any Lean-adjacent application. The glossary is not written for one of these readers at the expense of the others.

**On naming.** This project was originally developed and privately documented under the working title “The Mathlib Glossary Project.” I have since renamed it to **Ismail’s Glossary** to make explicit what was always true of the underlying work: this is an independent project built by one Lean user, not an official Mathlib or Lean FRO deliverable. The data, the descriptions, and the infrastructure are unchanged by the rename; only the framing is corrected.

## 2 Background: Lean 4, Mathlib4, and the Navigation Problem

### 2.1 What Lean 4 Is

Lean 4 is an interactive proof assistant and functional programming language developed by the Lean Focused Research Organization (Lean FRO) (de Moura and Ullrich, 2021). It is built on dependent type theory, in which mathematical statements are expressed as types and their proofs as programs; when a proof compiles, the Lean kernel certifies it as logically valid with machine precision — not approximately correct, not “looks right,” but checked to be free of logical error.

A common misconception is that Lean is a sophisticated typesetting system for mathematics — a more powerful  $\text{\LaTeX}$ . This gets the purpose backwards.  $\text{\LaTeX}$  asks only that symbols render correctly on a page; Lean asks whether the argument those symbols express is actually valid. It is closer to an unbiased logical judge than a typesetter: it follows the rules of mathematical reasoning with absolute precision, accepting no ambiguity, no hand-waving, and no implicit step. That distinction is why this glossary exists in the form it does: it is not documentation for a rendering tool, but a map of a rigorous, machine-checked mathematical universe.

## 2.2 Mathlib4 and Its Scale

Mathlib4 is the community-maintained mathematics library built on top of Lean 4 ([The mathlib Community, 2020](#)). It formalizes an extraordinary breadth of modern mathematics — abstract algebra, real and complex analysis, topology, number theory, algebraic geometry, category theory, combinatorics, probability theory, computability, and much more — organized as a hierarchy of directories and `.lean` source files whose structure mirrors the conceptual structure of mathematics itself.

Mathlib is not, however, the single largest library of formalized mathematics by total line count: the Mizar Mathematical Library, developed over several decades, remains larger, at roughly 3.7 million lines against Mathlib’s approximately 2.1 million ([Baanen et al., 2026](#)). What is well established is that Mathlib is among the largest and, by most accounts, the fastest-growing such library, and that it is the largest built on a dependently-typed proof assistant specifically ([Baanen et al., 2026](#)). That growth rate is itself part of the navigation problem this project addresses — not a footnote to it: a map that is accurate today needs an explicit, versioned relationship to the snapshot it describes, precisely because the territory keeps expanding (Section 10).

At approximately 9,150 addressable modules, Mathlib4 presents a genuine navigation challenge, and not only for newcomers. Even experienced Lean contributors regularly struggle to locate the right file for a given mathematical concept without resorting to text search or asking colleagues; for a mathematician exploring formalization for the first time, the library can be effectively opaque. The problem is structural, not a matter of any individual file being poorly written: there has simply been no high-level map saying, for instance, that spectral sequences live in a specific directory and build on the homological algebra infrastructure in a neighboring module.

## 2.3 Existing Tools and Their Limitations

Several tools exist for searching Mathlib, and each is genuinely useful for what it does — but none of them is a map, and confusing a search tool for a map is precisely the mistake this project is built to prevent ([Lean Community, 2025](#)).

| Tool                                                         | What It Does — and What It Cannot Do                                                                                                                                                                                                                                                                                                                                                  |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Loogle                                                       | Finds declarations by name or type signature (Breitner, 2023). Requires knowing roughly what you are looking for; gives no understanding of module organization or context.                                                                                                                                                                                                           |
| <code>exact?</code> / <code>apply?</code> / <code>rw?</code> | In-editor tactics that automatically close or rewrite a goal using an existing lemma — the successors to the retired <code>library_search</code> tactic. Each solves one tactical problem; none helps a user understand where to navigate.                                                                                                                                            |
| Moogle / LeanSearch / LeanExplore                            | Natural-language semantic search over Mathlib declarations. Still declaration-centric, with no concept of what a file or directory is about as a whole. Moogle pioneered this category but is, by the Lean community’s own account, no longer being actively updated by its maintainers (Lean Community, 2025); LeanSearch and LeanExplore are more recently maintained alternatives. |
| <code>#check</code> / <code>#find</code>                     | In-editor commands for exploring the type-checking environment. Requires already knowing a declaration name or type.                                                                                                                                                                                                                                                                  |
| Auto-generated docs                                          | HTML documentation built from source-code comments. Excellent for a module you already know; no discovery mechanism for one you do not.                                                                                                                                                                                                                                               |

Table 1: Existing Mathlib search tools and their scope.

Every tool in Table 1 presupposes that the user already knows, at least approximately, what they are looking for. Not one of them can answer the question that actually stalls a newcomer: *I want to work with spectral sequences — where in Mathlib should I begin, and what is there?* That is the question the glossary is built to answer.

### 3 What Was Built: Five Deliverables

The project produces five distinct, complementary deliverables, each serving a different audience or use case. All are hosted in a single public GitHub repository (Section 7).

#### 3.1 The Glossary JSON (Primary Data File)

The primary deliverable is a single structured JSON file containing all 9,150 entries. Every entry carries six fields:

| Field              | Description                                                                                                                           |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Path</b>        | Full module path from the Mathlib root — e.g. <code>Mathlib/Algebra/Homology/DerivedCategory</code>                                   |
| <b>Name</b>        | File or directory name alone — e.g. <code>DerivedCategory</code>                                                                      |
| <b>Type</b>        | <code>directory</code> for a folder containing further modules; <code>file</code> for a <code>.lean</code> source file                |
| <b>Parent Path</b> | The containing folder, used by automated tooling                                                                                      |
| <b>Depth</b>       | Nesting level: 0 = Mathlib root, 1 = top-level domain ( <code>Algebra</code> , <code>Analysis</code> , ...), 2 = subdomain, and so on |
| <b>Description</b> | One to three sentences: what the module contains, who uses it, how it relates to and differs from adjacent modules                    |

Table 2: Fields in the primary glossary JSON.

The JSON is versioned against the reference snapshot and formatted for direct consumption by any retrieval pipeline, vector database, or AI platform: load it as a JSON array, index the `Description` field

for semantic search, and use `Path`, `Type`, and `Depth` as metadata filters. Nothing about the format is Lean-specific; it is deliberately just a JSON array.

### 3.2 The RAG JSON (Embedding-Ready Export)

Alongside the primary JSON, the repository provides a companion RAG JSON — a flat, pre-processed export for developers building retrieval-augmented generation pipelines. The primary JSON already works for RAG as-is, since its entries are short, self-contained, and consistently structured; the RAG JSON removes the remaining preprocessing step. Each entry contains a single text field that merges the path, type, and description into one string ready for direct embedding, plus a metadata block recording the Lean version, Mathlib snapshot, and glossary version (Section 10.1) against which it was generated. This is the batteries-included option for anyone who wants a Mathlib-aware retrieval system running with minimal friction.

### 3.3 The Claude Skill

The Claude Skill is a self-contained bundle — a `.skill` archive — that installs the glossary as an active reference inside Claude. It contains three components: `SKILL.md`, a structured metadata file telling Claude when to activate the skill and how to use the search script; `references/glossary.json`, the full glossary embedded inside the bundle so the skill needs no external network access; and `scripts/search_glossary.py`, a Python search script that retrieves relevant entries by path prefix, keyword, module name, depth level, or type, rather than loading all 9,150 entries into context at once.

The bundled architecture is a deliberate choice, not an implementation convenience. An external reference introduces version drift, network failure, and link rot as live risks; a self-contained skill keeps working at inference time regardless of connectivity or later repository changes. Section 5 describes the skill and its anti-hallucination design in full.

### 3.4 The Master Spreadsheet

The master spreadsheet is the authoritative contributor workbook and the source from which every other deliverable is generated. It lives as an actively edited Google Sheet, linked from the repository, with two tabs: an **Instructions** tab — the complete contributor guide to column structure, writing-style guidelines, priority order, and status codes — and the **Ismail’s Glossary** tab itself, one row per module, carrying a **Status** column that marks each entry Complete, Pending, Needs Review, or Benchmark Theorem.

Because the live spreadsheet changes as the community contributes, the repository also publishes a periodic static `.xlsx` snapshot alongside the live link, so that anyone who needs a fixed, offline, or citable copy — rather than whatever the sheet says today — always has one (Section 7).

Contributions go through moderation before merging, and this is a requirement, not a formality: a glossary entry that misdescribes a module is worse than no entry at all, since it actively misleads the next reader. The Benchmark Theorem flag marks entries describing landmark formalized results, so those entries can carry enhanced description and visibility (Section 4.4).

### 3.5 The Interactive Website

The repository includes a fully functional HTML website intended for hosting on GitHub Pages, with five sections behind a top navigation bar: **Home**, an overview of the project and its audience; **Glossary**, an interactive, searchable tree of the entire Mathlib hierarchy, where hovering any node reveals its name, path, and description and clicking locks it for extended reading, each panel carrying a quick-copy control for the Lean import path or the local file path (Section 9.4); **Atlas**, a dedicated visual map of all 32 top-level domains surveyed to three layers of depth — built for orientation, not lookup, and treated in full in Section 9.3; **Lean 4 References**, a beginner-friendly syntax guide covering commands, tactics,



Unicode input, and common error messages, annotated with mathematical analogies for readers without Lean experience; and **Getting Started**, step-by-step environment setup for Windows and macOS/Linux.

Search draws from the human-readable descriptions rather than raw module names or paths, so a user can search using ordinary mathematical language — “homotopy equivalence,” “compact operators” — rather than needing to guess a Lean naming convention first. The glossary is findable the way a dictionary is findable: by what something means, not by how it is spelled in code.

## 4 The Glossary Data: Design and Coverage

### 4.1 Complete Coverage for the Reference Snapshot

The glossary is fully complete for the reference snapshot: every one of the 9,150 modules, across all six depth levels, carries a description. Table 3 gives the breakdown by depth and type, verified directly against the published JSON, RAG JSON, and spreadsheet exports, which agree exactly — not approximately, to the entry.

| Category                 | Count        | Share of total |
|--------------------------|--------------|----------------|
| Directories (all depths) | 1,129        | 12.3%          |
| Files — Depth 1          | 2            | <0.1%          |
| Files — Depth 2          | 892          | 9.7%           |
| Files — Depth 3          | 4,482        | 49.0%          |
| Files — Depth 4          | 2,327        | 25.4%          |
| Files — Depths 5–6       | 318          | 3.5%           |
| <b>Total</b>             | <b>9,150</b> | <b>100%</b>    |

Table 3: Coverage of Ismail’s Glossary by structural depth, reference snapshot 2026-05-09. Directories and files sum to  $1,129 + 8,021 = 9,150$ . Of the 9,150 entries, 9,107 carry Complete status and 43 carry Benchmark Theorem status (Section 4.4); none are Pending or Needs Review.

The 1,129 directory entries, at every depth, form the structural skeleton of the library: every folder at every level of nesting is described, which is what makes this a navigational map rather than a glossary of favorites. The file-level entries then add granularity, so a reader can look up exactly which definitions and lemmas live in a specific source file before opening it.

### 4.2 The ‘Distinct From’ Design Philosophy

One of the glossary’s most deliberate design features is systematic disambiguation. Mathlib contains many pairs — and sometimes clusters — of modules whose names are similar but whose purposes differ in ways not obvious from the names alone. Every description that needs one includes an explicit “distinct from  $X$ , which...” clause, because leaving the distinction implicit is exactly how a newcomer ends up importing the wrong module.

Consider `HomotopyCategory` and `DerivedCategory`, both nested under `Mathlib/Algebra/Homology`. Both live in homological algebra, both involve chain complexes, and both are central to modern algebraic topology and algebraic geometry — but the homotopy category quotients chain maps by chain homotopies, while the derived category goes further and localizes at quasi-isomorphisms. A newcomer importing the wrong one may spend hours debugging a proof before noticing the distinction; the glossary states it directly, in both entries, rather than leaving it to be discovered the hard way.

## Why This Matters

This mirrors how mathematicians actually navigate unfamiliar territory: not by reading one definition in isolation, but by understanding how a concept relates to — and differs from — its neighbors. A map that only names locations, without marking what separates them, is not yet a map.

### 4.3 Plain-Language Searchability

All descriptions are written in plain mathematical English, avoiding Lean-specific jargon wherever the underlying mathematics allows it. This is not a stylistic preference; it is what makes the glossary searchable by the terms a mathematician actually thinks in. A user who types “spectral sequences” finds the relevant modules regardless of how Lean happens to name them internally. A student who types “groups acting on sets” lands in the right part of the algebra hierarchy. The glossary is findable the way a dictionary is findable: by what a concept means, not by its identifier in the codebase.

### 4.4 Benchmark Theorem Entries

Entries flagged as Benchmark Theorems receive extended treatment. These are historically significant results formalized in Mathlib — the Monotone Convergence Theorem, the Sylow theorems, the Strong Law of Large Numbers, and the Fundamental Theorem of Algebra among them, 43 flagged entries in total, spanning analysis, algebra, number theory, and probability. Their entries carry not only a description of the module but context about the theorem’s mathematical significance, its statement, and its role in Mathlib’s dependency structure. This is not decoration: a reader encountering a theorem name for the first time should come away from the entry understanding why the result matters, not only where it lives.

## 5 The Claude Skill: Structured Retrieval for AI

### 5.1 The Problem It Solves

Large language models trained on Lean and Mathlib code have some familiarity with the library’s structure, but that knowledge is frozen at training time. A model trained before the reference snapshot has no knowledge of modules added or reorganized afterward, and it may produce confident, fluent, and wrong guidance about module locations, import paths, or theorem placements. This is not a failure of intelligence; it is a structural fact about how language models work. The fix is not to retrain the model more often, but to give it a reliable, current reference at inference time — exactly what a frozen model cannot supply for itself.

The Claude Skill does this directly. Rather than answering Mathlib navigation questions from training-time memory, Claude reads the glossary on demand. It does not inject all 9,150 entries into the context window — that would be wasteful and, past a point, counterproductive — but calls the search script for only the relevant slice, typically 5–20 entries, and reasons from those.

### 5.2 How the Skill Works

When the skill activates, Claude identifies the nature of the query and calls the search script with the appropriate parameters:

| Search Mode                                           | Use Case                                                         |
|-------------------------------------------------------|------------------------------------------------------------------|
| <code>--path "Mathlib/X/Y"</code>                     | Get the entry for a specific path and optionally its children    |
| <code>--keyword "homotopy"</code>                     | Search names and descriptions for a concept or term              |
| <code>--name "DerivedCategory"</code>                 | Look up a specific module by name (partial match)                |
| <code>--depth 1 --type directory</code>               | Get all top-level domains — a structural overview of the library |
| <code>--path "Mathlib/Algebra/Lie" --type file</code> | List all source files within a subdomain                         |
| <code>--keyword "spectral" --max 30</code>            | Broader search with an increased result limit                    |

Table 4: Search modes exposed by `scripts/search_glossary.py`.

Results return as structured text, and Claude synthesizes a response from the descriptions: which path to look at, what it contains, how it relates to adjacent modules, and — for disambiguation questions — a side-by-side comparison drawn from both entries at once.

### 5.3 Anti-Hallucination by Design

The skill exists, specifically, to prevent hallucinated Mathlib references — a real and consequential failure mode for any AI tool used in Lean development, not a theoretical one. A language model generating an import path or a lemma name from memory can produce something plausible and wrong: a path that does not exist, a lemma with a subtly incorrect name, a module reorganized in a later release. In a compiled proof assistant these failures surface eventually — but only after the user has already been misled and has already lost the time.

Grounding responses in the glossary rather than in training memory means Claude’s Mathlib guidance is accurate to the reference version and explicit about what it actually found. The skill also instructs Claude to say so when it finds nothing relevant, which is the correct behavior: a confident “I don’t know” costs less than a confident wrong answer.

### 5.4 Installation and Usage

The skill is distributed as a `.skill` archive in the GitHub repository. Installing it in Claude is a one-time upload through the Skills interface; after that, it activates automatically whenever a query involves Mathlib navigation — where to find a theorem, what a module contains, which import to use, or how two adjacent modules differ.

#### Example Queries

“Where is the Hahn–Banach theorem in Mathlib?” “What is in Mathlib/Analysis/SpecialFunctions?” “What is the difference between `HomotopyCategory` and `DerivedCategory`?” “I want to work with Haar measure — where do I start?”

## 6 AI and LLM Integration: Beyond Claude

The Claude Skill serves Claude specifically; the broader ambition is a reliable Mathlib reference for any AI platform, coding assistant, or proof tool that could benefit from structured knowledge of the library’s organization.

### 6.1 The Open JSON on GitHub

The primary JSON and the RAG JSON are published openly under the terms in Section 8. Any developer, research team, or AI company can download, host, embed, index, or train on them. Any LLM-powered tool that needs to reason about Mathlib — a proof assistant, a documentation search engine, a mathematics

tutoring system — can have a clean, reliable, versioned data source without scraping or parsing the Mathlib source directly.

This is, in effect, search-engine indexing for a library that never had any: it does not change what Mathlib contains, only what of it a machine can find.

## 6.2 Retrieval-Augmented Generation

The most immediate AI use case is retrieval-augmented generation (RAG): a system indexes text in a vector database and, on a query, retrieves the most semantically similar entries to inject into the model’s context, so the model answers from what was retrieved rather than from memory alone.

The glossary is structured for exactly this. Each entry is short, semantically self-contained, and scoped to a single well-defined module — properties that make it an efficient RAG corpus: fast retrieval, compact injected context, and a clean answer to a specific navigational question. A query about the Central Limit Theorem retrieves the relevant probability and analysis entries, which the model uses to answer precisely rather than plausibly. The companion RAG JSON (Section 3) removes the last preprocessing step, so a Mathlib-aware assistant can be running in minutes rather than days.

## 6.3 Fine-Tuning and Instruction Datasets

Beyond retrieval, the completed glossary is a fine-tuning corpus with no additional curation required. Each entry converts into an instruction-response pair for free: the prompt is “What does `Mathlib/X` contain?” and the response is the description. Nine thousand such pairs, drawn from a single carefully checked source, are a meaningful dataset for any model intended to assist with Lean development, formalization, or formal-methods research.

## 6.4 Integration with AI Proof Assistants

Tools that put language models directly inside the Lean proof environment — LeanCopilot (Song et al., 2025) among them — represent the frontier of AI-assisted formalization. As these mature, generating syntactically plausible Lean code will not be enough; they will need structured knowledge of the library to reason about which modules to import, which lemmas already exist, and where to look when a proof stalls. The glossary is built to be that reference layer.

# 7 GitHub Repository Structure

All deliverables are organized in a single public GitHub repository, `IsmailsGlossary`, structured so a first-time visitor understands immediately what each file is for.

| File / Directory             | Purpose                                                                                             |
|------------------------------|-----------------------------------------------------------------------------------------------------|
| README.md                    | Project overview, quick-start guide, version information, contribution instructions                 |
| IsmailsGlossary.json         | The full 9,150-entry glossary — primary data file for AI tools, RAG pipelines, and general use      |
| IsmailsGlossaryRAG.json      | Flat, embedding-ready export for retrieval-augmented generation pipelines                           |
| ismails-glossary.skill       | The Claude Skill bundle — install this in Claude for Mathlib navigation assistance                  |
| IsmailsGlossary_vMASTER.xlsx | Static snapshot of the master spreadsheet; the live, actively edited copy is linked from the README |
| index.html (GitHub Pages)    | The interactive website: glossary tree, Atlas, Lean 4 reference, and getting-started guide          |
| LICENSE-DATA, LICENSE-CODE   | License texts for the data/documentation and code components respectively (Section 8)               |

Table 5: Top-level layout of the `IsmailsGlossary` repository.

The repository is pinned to the reference snapshot: Lean 4.29.1, Mathlib4 commit `1ad783f9bf`. Anyone who wants that exact baseline can fork it at this point directly. Later updates are tagged with their corresponding glossary version (Section 10.1), so the tag history is a complete, checkable audit trail of how the glossary has evolved alongside the library it describes.

## 8 Licensing, Citation, and Availability

### 8.1 License

Ismail’s Glossary is released under a split license, because its two kinds of content are not the same kind of thing. **Documentation and data** — the glossary JSON, the RAG JSON, the spreadsheet, the website content, and this paper — are released under Creative Commons Attribution 4.0 International (CC-BY-4.0): anyone may copy, redistribute, remix, and build on this material for any purpose, including commercially, provided appropriate credit is given. **Code** — the search script, the skill’s supporting logic, and the website’s HTML/CSS/JavaScript — is released under the MIT License: anyone may use, copy, modify, and redistribute it with minimal restriction, provided the license notice is retained.

This split is not idiosyncratic; it is standard practice for a project that combines reference data with software tooling, and both full license texts are included in the repository (`LICENSE-DATA`, `LICENSE-CODE`).

### 8.2 How to Cite This Work

Please cite the Zenodo record for the version used, so citations remain tied to a specific, immutable snapshot of the data rather than to whatever the live repository happens to contain when read:

Ismail, M. (2026). Ismail’s Glossary: A Complete Navigation Index for Mathlib4 (v4.30) [Data set]. Zenodo. <https://doi.org/10.5281/zenodo.21192789>

```
@misc{ismail2026glossary,
  author = {Ismail, Muhammed},
  title = {Ismail’s Glossary: A Complete Navigation Index for Mathlib4},
  year = {2026},
  publisher = {Zenodo},
  version = {v4.30},
  doi = {10.5281/zenodo.21192789},
  url = {https://doi.org/10.5281/zenodo.21192789}
}
```

*This DOI has been reserved on Zenodo for this record and will resolve once the deposit is finalized and published.*

### 8.3 Availability

The GitHub repository (<https://github.com/M-Ismail-ZA/IsmailsGlossary>) hosts all five deliverables, versioned history, and the interactive site via GitHub Pages. This paper and a versioned copy of the data are additionally archived on Zenodo under the DOI above, so the record’s long-term preservation and citability do not depend on GitHub’s continued availability. The master spreadsheet (Section 3.4) is maintained as a live Google Sheet linked from the repository README; the repository carries a periodic static snapshot for anyone who needs a fixed, offline, or citable copy rather than the current live state.

## 9 Accessibility and Educational Philosophy

### 9.1 Designed for All Skill Levels

The glossary is written for readers at every level of mathematical and technical sophistication, from a curious high-school student to an experienced mathematician encountering Lean for the first time. Every entry follows the same rule: plain English, standard mathematical notation rather than Lean syntax, no assumed familiarity with proof assistants.

This is a deliberate departure from most Lean documentation, which tends to assume the reader is already comfortable with dependent type theory, already knows what a typeclass is, and can already read Lean’s terse declaration syntax. That assumption is a real barrier to adoption, not a minor inconvenience. The glossary removes it: a reader who understands what a topological space is can read the entry for `Mathlib/Topology` and come away knowing what is there and why it matters, without knowing anything about how Lean represents it internally.

### 9.2 Formalization Literacy as Early Education

One longer-term goal is to make mathematical formalization accessible at the secondary-school level. The reasoning is direct: calculus is already taught before university, and a student who can differentiate, follow a proof, and reason logically already has the foundation needed to engage with formal mathematics — provided the tools are written for them and not for graduate students by default.

Making the glossary readable from high-school level serves a broader goal: a student who has already met Lean through the glossary’s plain explanations should not find formal verification alien or intimidating on first contact at university. The map should come before the territory, not after.

### 9.3 The Mathematical Divergence Perspective

The interactive site offers something no other Mathlib tool provides: a visual picture of how mathematical specialties diverge from common roots, shown from two complementary angles. The **Glossary** tab is built for lookup — every module traces a path from the root through increasingly specialized territory, and hovering or clicking any node surfaces its full description. The **Atlas** tab, added after the glossary’s initial release, is built for orientation instead — a dedicated visual map of all 32 top-level domains surveyed to three layers of depth, meant to be explored before a reader knows exactly what they are looking for.

The two views serve one idea from different directions. A statistician working in probability sits, in the library’s actual structure, closer to an algebraist working in measure theory than either might expect, and both share foundations with the analyst studying Lebesgue integration; the Atlas makes that shape visible at a glance, while the Glossary tree lets a reader follow any single branch down to the file level.

## Cross-Domain Perspective

A statistician and a logician may feel their fields are worlds apart — but browse the Atlas and the picture changes: both descend from the same root, diverging at a specific, nameable branch. The glossary makes that moment of divergence visible, not merely assumed.

This has educational value that has nothing to do with Lean. A researcher trained in one specialty who opens the glossary can see, for the first time, exactly where their field diverges from the neighboring ones, and exactly what ground they still share. Mathlib’s breadth is a navigation problem in Section 2.2; here, rendered as a browsable map, the same breadth becomes an asset: a structured survey of the entire landscape of modern formalized mathematics.

### 9.4 Quick Copy: From Reading to Proving

One practical feature reflects the glossary’s dual nature as reference and working tool. Every module description in the interactive glossary includes a quick-copy control with two options: the Lean import path, ready to paste into a `.lean` file, or the local filesystem path, ready for a terminal or file browser. A reader does not only learn what a module contains; they can act on that immediately, with no separate lookup step in between. Understanding and using are one click apart, not two.

## 10 Community Model: A Living Document

### 10.1 Versioning: Three Numbers, Not One

Three distinct identifiers run through this project’s data and tooling, and conflating them is an easy, natural mistake if they are not stated explicitly. The **Lean toolchain version** (e.g. 4.29.1) is the version of the Lean 4 compiler and elaborator itself, following Lean’s own release cadence. The **Mathlib snapshot** (e.g. 2026-05-09, commit 1ad783f9bf) is the specific state of the Mathlib4 repository the glossary describes — the authoritative reference point, since two glossaries can both cite “Lean 4.29.1” and still describe different Mathlib states, as Mathlib keeps committing while remaining compatible with a given toolchain. The **glossary version** (e.g. v4.30) is Ismail’s Glossary’s own release counter, incremented independently each time the glossary is updated, whether or not the underlying Mathlib snapshot changes.

The glossary version number does *not* correspond to any Mathlib or Lean release carrying the same numeral, and the two should never be read as interchangeable: Mathlib’s own v4.30.0 tag tracks Lean 4.30.0, released 2026-05-26 — a distinct and later point than this glossary’s reference snapshot. Every version claim in this paper and in the project’s data files keeps these three numbers separate for exactly this reason.

### 10.2 Complete at the Reference Snapshot

The glossary is fully complete for its reference snapshot, and that word is meant literally: this is not a partially filled work in progress that happens to be public. It launches finished — every module described, every entry reviewed, every path checked against the Mathlib source — and the reference snapshot is a stable foundation a user can rely on today, specified precisely by the three numbers in Section 10.1.

As of this paper’s writing, Lean has advanced through 4.30.0, 4.31.0, and a 4.32.0 release candidate since the glossary’s 2026-05-09 snapshot — roughly two months and three releases. That gap is by design, not oversight: the reference snapshot is a deliberately pinned, citable baseline (Section 10.3), and the project’s living-document model (Section 10.4) is how it stays useful as Mathlib continues past it.



### 10.3 Forking for Exact Reproducibility

Anyone who needs a fully stable reference — a reproducible build, a locked dependency, a shared baseline across a team — can fork the repository at the reference snapshot tag and get an exact, immutable copy of both Mathlib and the glossary as they stood on 2026-05-09. Updates continue on the main branch; a fork at commit `1ad783f9bf` is a permanent anchor that will never move under anyone.

### 10.4 Contribution Workflow

Community contributions run through the master spreadsheet (Section 3.4). It is openly accessible: anyone with mathematical knowledge of a Lean module can propose a description or improve an existing one. Contributions are moderated before approval, and this is quality assurance, not gatekeeping — the precision standard has to be high, because a description that is mathematically imprecise, that conflates two adjacent modules, or that would mislead a newcomer is actively worse than leaving the entry blank.

Spreadsheet comments stay open for discussion: a contributor can leave notes or questions alongside a proposed description, and a moderator can respond before approving it. That leaves a paper trail for any entry whose wording was genuinely deliberated, which future contributors can consult instead of re-litigating the same question.

### 10.5 Mathlib Grows, the Glossary Follows

Mathlib is under active development: new modules arrive with every release, and existing ones are occasionally reorganized, renamed, or split. The glossary is built to track this — new entries follow the same structure and writing standard as existing ones, and the version tags in Section 10.1 make it unambiguous which state of the library a given glossary version describes. My own ongoing formalization work is a continuous field test in the most literal sense: using the glossary in practice surfaces gaps and inaccuracies that no amount of review from the outside would have caught first.

## 11 Significance and Impact

### 11.1 For Mathlib Contributors and Users

The glossary lowers the barrier to Mathlib contribution measurably, not just in spirit. A mathematician who wants to contribute a formalization currently has to reconstruct the library’s structure from scratch — which modules exist, which definitions are already present, where a new file belongs, which import chain already applies — before writing anything. A complete glossary turns that orientation phase from hours into minutes: search for adjacent work, understand the surrounding context, find the right place for the contribution, all before the first line of code.

### 11.2 For Mathematics Education

Independent of any Lean-specific purpose, the glossary is a structured survey of the landscape of modern formalized mathematics. A student wondering what algebraic K-theory is, how measure theory relates to probability, or what model theory has to do with algebraic geometry can browse the glossary as a guided answer to exactly that curiosity. The library’s hierarchy mirrors the hierarchy of mathematics itself, and every entry is written to inform a reader without specialist background — not to gatekeep one who lacks it.

This is not incidental to the project. The glossary is meant to work as a gateway: a resource that makes advanced mathematics feel navigable and connected, rather than isolated and intimidating, at precisely the point where a student is deciding whether to go further.



### 11.3 For AI-Assisted Formalization

As AI tools become more deeply embedded in formal mathematics workflows, demand for structured, machine-readable Mathlib metadata will only grow. The glossary is a direct answer to that demand: a clean, consistently formatted, versioned description of every module, available in forms meant for both human and machine readers. Any team building Lean-adjacent AI tools — a proof assistant, a documentation engine, a formalization advisor, an educational agent — gets, in the completed glossary, a ready-made knowledge base that would otherwise take months to produce from nothing.

### 11.4 For Independent Research

The project is also a demonstration of a model for independent scholarship in formal mathematics: identify a genuine infrastructure gap, build a systematic answer to it, and release something the whole community can use, without waiting for institutional backing to do so. This kind of infrastructure work is routinely undervalued relative to proving new theorems, but it is what makes a growing library navigable at all — and navigability, not raw size, is what decides whether formalized mathematics stays the province of specialists or becomes a broadly usable scientific tool.

## 12 Conclusion

Ismail’s Glossary fills a specific, long-standing gap in the Lean 4 and Mathlib ecosystem: a complete, human-readable navigation guide for one of the largest libraries of machine-checked mathematics in existence — 9,150 modules, fully described, fully versioned, and released as an independent community resource, not an official one.

The project is useful at every level of engagement, not just the most technical. A high-school student curious about the reach of modern mathematics can browse the Atlas and see exactly where calculus ends and topology, probability, and algebra begin. A graduate student starting a first formalization project can find the right import path in seconds instead of hours. An AI researcher building a Lean-aware tool can drop the JSON into a retrieval pipeline and have an immediately hallucination-resistant knowledge base for Mathlib queries. An experienced Lean contributor can navigate unfamiliar parts of the library with a map that, for the first time, actually reflects its full extent.

The project is complete for its reference version and built to keep growing: forking the repository pins an exact baseline; the main branch tracks Mathlib forward; community contributions run through a moderated workflow with the master spreadsheet as the single source of truth. The Claude Skill makes the glossary available to Claude at inference time; the open JSON makes it available to everyone else.

#### Closing Thought

Understanding a module’s description means learning something about mathematics and discovering where that mathematics lives, formally verified, at the same time. The two are not separable by design; that was the point of writing it this way.

When asked where to begin with Lean 4 and Mathlib, the honest answer used to be: somewhere, if you can find it. With Ismail’s Glossary, the answer is a map, not a shrug.

## References

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, Lecture Notes in Computer Science, vol. 12699, pages 625–635. Springer, Cham, 2021. doi:[10.1007/978-3-030-79876-5\\_37](https://doi.org/10.1007/978-3-030-79876-5_37)

The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs* (CPP 2020), New Orleans, LA, USA, pages 367–381, 2020. doi:[10.1145/3372885.3373824](https://doi.org/10.1145/3372885.3373824)

Anne Baanen, Matthew R. Ballard, Johan Commelin, Bryan Gin-gé Chen, Michael Rothgang, and Damiano Testa. Growing Mathlib: maintenance of a large scale mathematical library. In *Intelligent Computer Mathematics* (CICM 2025), Lecture Notes in Computer Science, vol. 16136. Springer, Cham, 2026. doi:[10.1007/978-3-032-07021-0\\_4](https://doi.org/10.1007/978-3-032-07021-0_4)

The Lean Community. Searching for theorems in Mathlib. *Lean community blog*, 2025. <https://leanprover-community.github.io/blog/posts/searching-for-theorems-in-mathlib/>

Joachim Breitner. Loogle: a search engine for Lean and Mathlib. Lean Focused Research Organization, 2023. <https://loogle.lean-lang.org/>

Peiyang Song, Kaiyu Yang, and Anima Anandkumar. Lean Copilot: large language models as copilots for theorem proving in Lean. In *Proceedings of the International Conference on Neuro-symbolic Systems* (NeuS 2025), Proceedings of Machine Learning Research, vol. 288, pages 144–169, 2025.